

UAX FP

Universidad
Alfonso X el Sabio



Rehab&Move

Aplicación enfocada al paciente para mejorar la adherencia
en la rehabilitación de fisioterapia.

2º Desarrollo de Aplicaciones Multiplataforma /
Modalidad Online

Alumno: David Prados Medina

Tutor: Damián Sudea Soy



DEDICATORIA

Querría agradecer a mis amigos Kike y Mario, son grandes profesionales de este sector y me han ayudado mucho con sus consejos en este proyecto.

También agradecer a Lucía, mi motor en todos mis retos, ella hace que todo sea siempre más sencillo.



ÍNDICE

ABSTRACT	6
JUSTIFICACIÓN DEL PROYECTO	7
INTRODUCCIÓN	9
OBJETIVO.....	11
R01: Gestión de fisioterapeutas.....	11
R01F01: Registrar fisioterapeutas en el sistema.	11
R01F02: Autenticación de fisioterapeutas.	11
R02: Gestión de pacientes.....	12
R02F01: Registrar pacientes asociados a un fisioterapeuta.....	12
R02F02: Autenticación de pacientes.	12
R03: Control de Acceso y Roles.	13
R03F01:Restringir funciones según el tipo de usuario.	13
R04: Gestión de citas terapéuticas.	13
R04F01: Registrar citas entre fisioterapeuta y paciente.	13
R05: Registro de dolor (Escala EVA).....	15
R05F01: Registro del nivel del dolor por parte del paciente.	15
R06: Gestión de ejercicios terapéuticos.....	16
R06F01:Crear y consultar ejercicios disponibles.	16
R07: Asignación de ejercicios a pacientes.....	16
R07F01:Asociar ejercicios mediante planificación terapéutica.	16
R08: Registro de ejercicios realizados.....	17
R08F01: Registrar la ejecución de ejercicios asignados.....	17
R09: Arquitectura cliente - servidor.....	17
R09F01: Implementar API REST mediante FastAPI.	17
R09F02: Interfaz gráfica de escritorio para el fisioterapeuta (PySide6).....	18
R09F03: Interfaz Web para el paciente (React + TypeScript).	18
R10: Infraestructura y Despliegue Cloud.	19
R10F01: Despliegue en Amazon Web Services (AWS).....	19
R11: Automatización y CI/CD (GitHub Actions).	20
R11F01: Implementar un flujo de trabajo de integración y despliegue continuo.	20
DESCRIPCIÓN	21
Arquitectura Cliente - Servidor.	21
Caso de Uso: Asignar un ejercicio a un paciente.	25



DISEÑOS	27
Diagrama Entidad - Relación.	27
Entidad 'Physio'.	28
Entidad 'Patient'.	28
Entidad 'Appointment'.	28
Entidad 'Pain_record'.	29
Entidad 'Exercise'.	29
Entidad 'Exercise_done'.	29
Entidad intermedia 'Exercise_assignment'.	29
Diagrama de la base de datos.	30
Logo.	31
Interfaces de la aplicación: Aplicación de escritorio.	31
Login.	31
Dashboard del fisioterapeuta.	32
Ventanas emergentes de añadir y modificar paciente.	32
Calendario de sesiones (fisioterapeuta).	33
Ventanas emergentes de visualizar sesiones diarias y añadir citas.	33
Registro de dolor de un paciente.	34
Ventana emergente con detalle de un registro.	34
Menú de ejercicios.	35
Ventana emergente con información del ejercicio.	35
Ventanas emergentes de añadir, modificar y asignar un ejercicio.	36
Menú de ejercicios asignados.	36
Ventanas emergentes de asignar ejercicio y editar plan de ejercicio.	37
Interfaces de la aplicación: Aplicación web.	38
Login.	38
Dashboard del paciente.	38
Página Mis Ejercicios.	39
Ventanas emergentes en Mis Ejercicios.	39
Página Mis Citas.	40
Página Registro de Dolor.	40
Ventanas emergentes de Registro de Dolor.	41
Página Registro de Dolor (Registro hecho).	41
TECNOLOGÍA.	42
METODOLOGÍA.	50



Planificación temporal inicial.	51
Planificación temporal real.	52
Análisis de las desviaciones.	53
Diagrama de Gantt.	54
VIABILIDAD DE NEGOCIO	55
Análisis de mercado y público objetivo	55
Análisis DAFO	55
Modelo de negocio: Monetización	56
Viabilidad técnica y económica	57
Viabilidad técnica	57
Viabilidad económica	58
Conclusión de Viabilidad de Negocio	59
TRABAJO FUTURO	60
CONCLUSIONES	62
ENLACES	63
Enlace a GitHub.	63
Enlace a YouTube	63
REFERENCIAS	64



ABSTRACT

Rehab&Move es una aplicación que surgió para resolver un problema en específico: la falta de adherencia de los pacientes a la hora de realizar ejercicios desde casa.

Esta aplicación multiplataforma permite: asignar ejercicios específicos a pacientes, gestionar citas, gestionar pacientes, ver la realización de los ejercicios asignados y observar la evolución del dolor.

Dividiéndose en dos bloques de frontend: aplicación web (React + TypeScript) y aplicación de escritorio (PySide 6 + Python) **Rehab&Move** obtiene su información de manera sincronizada gracias a un backend común (FastAPI + Python) desplegado en la nube (AWS).

Este proyecto me ha permitido comprobar que un desarrollo es muy satisfactorio e incluso mucho más efectivo, cuando el programador comprende profundamente el sector en el cual está trabajando. Mi experiencia previa me ha permitido realizar una aplicación que solventa los problemas que vivía en mi día a día.

***Rehab&Move** is an application that emerged to solve a specific problem: patient's lack of adherence to at-home exercise routines.*

This cross-platform application allows users to: assign specific exercises to patients, manage appointments, manage patients, view the completion of assigned exercises and monitor pain progression.

*Divided into two frontend components—a web application (React + TypeScript) and a desktop application (PySide 6 + Python)—**Rehab&Move** obtains its data synchronously thanks to a common backend (FastAPI + Python) deployed in the cloud (AWS).*

This project has allowed me to verify that development is highly satisfying and even more effective when the programmer has a deep understanding of the sector in which they are working. My previous experience enabled me to create an application that solves the problems I encountered daily.



JUSTIFICACIÓN DEL PROYECTO

La razón principal por la que decidí hacer este proyecto surge de mi experiencia profesional en el ámbito de la fisioterapia. Antes de estudiar el grado superior de Desarrollo de Aplicaciones Multiplataforma (DAM), ejercía como fisioterapeuta, en esta profesión observé como las herramientas tecnológicas eran de gran ayuda para los profesionales sanitarios siendo un apoyo sustancial en los procesos de rehabilitación. Además, diversos estudios, como el desarrollado por el terapeuta ocupacional Rodrigo Cubillos-Bravo, demuestran que aplicaciones interactivas, videojuegos terapéuticos o sistemas de realidad virtual mejoran la recuperación funcional de los pacientes (Rodrigo Cubillos-Bravo, 2022).

Durante la práctica clínica encontré un problema común: la falta de adherencia de los pacientes a la realización de los ejercicios terapéuticos pautados para realizar en su domicilio. Una parte fundamental de la recuperación funcional es el trabajo que hacen los pacientes desde casa, su constancia y la correcta realización de los ejercicios, sin embargo, la falta de supervisión profesional fuera de la consulta provocaba frecuentemente abandonos del tratamiento o ejecuciones incorrectas de los ejercicios.

Con el objetivo de resolver este problema y unir el ámbito sanitario y tecnológico, ya que son dos mundos que me apasionan, surge el desarrollo de la aplicación **Rehab&Move**, una aplicación multiplataforma enfocada a pacientes en proceso de rehabilitación que tienen que realizar ejercicios terapéuticos de forma autónoma en su domicilio.

La aplicación permitirá al fisioterapeuta asignar ejercicios terapéuticos junto a vídeos explicativos, lo que favorecerá la correcta ejecución por parte de los pacientes. Además, añade la funcionalidad de que el usuario pueda registrar la realización de ejercicios de forma diaria, mejorando la constancia y el seguimiento del tratamiento.



Como complemento al seguimiento de los ejercicios, la aplicación incorpora un sistema de registro del dolor mediante la escala EVA (Escala Visual Analógica), donde el paciente indica su nivel de dolor, del 1 al 10, tras la realización de los ejercicios junto a un comentario que genera un feedback personalizado para el profesional. Estos datos serán representados en una gráfica, lo que favorece tanto al paciente como al profesional la visualización de la evolución clínica de la lesión.

Asimismo, el sistema añadirá un calendario de seguimiento donde se mostrarán las citas programadas y las sesiones realizadas, esto tiene como finalidad la organización del tratamiento y la adherencia terapéutica.

Actualmente, existen aplicaciones enfocadas al ejercicio físico y al bienestar general en el mercado, incluso algunas enfocadas a la rehabilitación. Sin embargo, la mayoría de estas herramientas no proveen de supervisión profesional directa ni de personalización para procesos de recuperación de lesiones. Tampoco estas aplicaciones tienen una asociación directa con la gestión de pacientes en clínica, estando más enfocadas a la realización de ejercicios con vídeos genéricos. **Rehab&Move** ofrece al mercado un seguimiento clínico individualizado y una personalización más exhaustiva.

El público al que va dirigido esta aplicación son pacientes en proceso de recuperación tras lesiones musculoesqueléticas o neurológicas y profesionales de la fisioterapia que deseen hacer un seguimiento de forma remota a sus pacientes.

En definitiva, **Rehab&Move** ofrece ser una aplicación útil que pretende aportar valor tanto a profesionales como a pacientes, todo ello mediante la combinación de personalización profesional, seguimiento del dolor, control de la adherencia y organización del tratamiento de rehabilitación en una única plataforma digital.



INTRODUCCIÓN

Este proyecto final consiste en desarrollar la aplicación **Rehab&Move**, una aplicación multiplataforma que tiene como finalidad mejorar el seguimiento y la adherencia de la parte de rehabilitación que realizan los pacientes fuera de la consulta clínica. Esta herramienta busca favorecer la eficacia del tratamiento y, asimismo, mejorar la comunicación entre el paciente y el fisioterapeuta.

El problema principal que aborda este proyecto es la falta de constancia junto a las ejecuciones incorrectas de los ejercicios que realizan los pacientes desde casa, debido a su vez por la falta de guía y de supervisión profesional mientras se realizan dichos ejercicios. Esta situación puede conllevar abandonos de tratamientos o ineficacia de los ejercicios terapéuticos, lo que puede afectar negativamente a la recuperación.

La solución que ofrece esta aplicación es incorporar diferentes funcionalidades enfocadas en el seguimiento del tratamiento fisioterápico.

Entre las principales funciones de **Rehab&Move** se encuentran:

- Asignación de ejercicios adaptados a cada paciente con un vídeo explicativo.
- Registro de los ejercicios realizados.
- Control y visualización de la evolución del dolor a través de la escala EVA.
- Un calendario con las citas futuras y ya realizadas en clínica.



Con el fin de favorecer un correcto funcionamiento de la aplicación, el proyecto deberá cumplir los siguientes requisitos:

- Permitir la gestión de usuarios, diferenciando entre profesional sanitario y paciente.
- Facilitar la asignación y visualización de ejercicios terapéuticos.
- Registrar la realización del ejercicio por parte del paciente.
- Permitir realizar un seguimiento de la evolución del dolor.
- Mostrar información del dolor mediante una representación gráfica.
- Visualizar citas y sesiones de tratamiento.
- Ofrecer una interfaz intuitiva y sencilla para cualquier tipo de usuario.
- Garantizar la seguridad de los datos almacenados.

Para finalizar, **Rehab&Move** debe de ser una aplicación intuitiva con el objetivo de favorecer el proceso de recuperación funcional gracias al uso de la tecnología desde distintos dispositivos.

OBJETIVO

La aplicación **Rehab&Move** tiene como objetivo favorecer el seguimiento del proceso de rehabilitación de los pacientes a través de: facilitar herramientas para hacer los ejercicios terapéuticos, mantener un seguimiento de la evolución del dolor y permitir una gestión de los pacientes gracias a un calendario integrado. Todo esto se realiza gracias a una arquitectura basada en base de datos de PostgreSQL, API REST realizada en python a través de la librería FastAPI y una interfaz gráfica desarrollada en python también con la librería pySide6.

A continuación, voy a utilizar la metodología RTFP (Requisitos, Funciones, Tareas, Pruebas) con la finalidad de desglosar cada objetivo de este proyecto y generar un enfoque estructurado de los retos que van a surgir en el desarrollo de la aplicación.

R01: Gestión de fisioterapeutas.

R01F01: Registrar fisioterapeutas en el sistema.

- **R01F01T01:** Utilizar la tabla '*physio*' en la base de datos de PostgreSQL.
 - R01F01T01P01: Insertar fisioterapeutas mediante pgAdmin4.
- **R01F01T02:** Implementar endpoint REST a través de FastAPI.
 - R01F01T02P01: Creación correcta de fisioterapeuta mediante petición POST.

R01F02: Autenticación de fisioterapeutas.

- **R01F02T01:** Validar correo electrónico y contraseña almacenado en la base de datos.
 - R01F01T01P01: Permitir únicamente el acceso con las credenciales correctas.



R02: Gestión de pacientes.

R02F01: Registrar pacientes asociados a un fisioterapeuta.

- **R02F01T01:** Utilizar la tabla '*patient*' con relación a la tabla '*physio*'.
 - R02F01T01P01: Insertar fisioterapeutas mediante pgAdmin4.
- **R02F01T02:** Crear endpoint REST a través de FastAPI.
 - R02F01T02P01: Creación correcta de paciente mediante petición POST.
- **R02F01T03:** Añadir formulario de registro en la interfaz gráfica.
 - R02F01T03P01: Validación de campos obligatorios.
 - R02F01T03P02: Enviar datos al endpoint REST de registro.
 - R02F01T03P03: Confirmación visual de registro exitoso.

R02F02: Autenticación de pacientes.

- **R02F02T01:** Validar correo electrónico y contraseña almacenada en la BD.
 - R02F02T01P01: Permitir el acceso con las credenciales correctas.

R03: Control de Acceso y Roles.

R03F01: Restringir funciones según el tipo de usuario.

- **R03F01T01:** Definir roles a través del campo *'role'* de la base de datos (físio/paciente).
 - R03F01T01P01: Verificar el rol del usuario logueado.
- **R03F01T02:** Implementar la lógica de visibilidad en la interfaz gráfica.
 - R03F01T02P01: Ocultar botones y funcionalidades a usuarios que no sean fisioterapeutas.

R04: Gestión de citas terapéuticas.

R04F01: Registrar citas entre fisioterapeuta y paciente.

- **R04F01T01:** Utilizar tabla *'appointment'* con estados y relación paciente-físio.
 - R04F01T01P01: Inserción de citas con su estado inicial *'pendiente'*.
- **R04F01T02:** Crear endpoints para creación y actualización de su estado en FastAPI.
 - R04F01T02P01: Generación de cita inicial mediante petición POST.
 - R04F01T02P02: Cambio de estado a completado o cancelado vía PUT/PATCH.
 - R04F01T02P03: Eliminar cita a través de la petición DELETE.



- **R04F01T03:** Implementar un módulo de calendario y citas en la interfaz gráfica.
 - R04F01T03P01: Visualización de citas programadas por fecha en la aplicación de escritorio.
 - R04F01T03P02: Visualización de citas programadas por fecha en la web.

- **R04F01T04:** Implementar panel de gestión de agenda en la interfaz gráfica (PySide6). Disponible solo para usuario '*physio*'.
 - R04F01T04P01: Formulario para agendar nuevas citas seleccionando paciente y fecha.
 - R04F01T04P02: Botones eliminar cita desde el calendario.
 - R04F01T04P03: Refresco de la vista tras la modificación.

- **R04F01T05:** Mejora de la experiencia de usuario (UX) mediante selectores dinámicos.
 - R04F01T05P01: Implementación de QComboBox con búsqueda predictiva para evitar errores de inserción de ID manuales.
 - R04F01T05P02: Carga asíncrona desde la API al abrir los diálogos.



R05: Registro de dolor (Escala EVA).

R05F01: Registro del nivel del dolor por parte del paciente.

- **R05F01T01:** Uso de la tabla 'pain_record' y registro del nivel de dolor.
 - R05F01T01T01: Validación de números enteros del 1 al 10.
- **R05F01T02:** Implementar endpoint REST para registro en FastAPI.
 - R05F01T02T01: Recepción y almacenamiento del registro en la marca del tiempo(timestamp).
- **R05F01T03:** Añadir slider en la interfaz gráfica de la web (para los usuarios 'patient').
 - R05F01T03T01: Envío del valor EVA asociado al ID del paciente.
 - R05F01T03T02: Confirmación visual de registro exitoso.
- **R05F01T04:** Representación visual en la interfaz gráfica de la evolución del dolor (Matplotlib).
 - R05F01T04T01: Implementar el gráfico en el módulo de registro de dolor de la aplicación de escritorio.
 - R05F01T04T02: Implementar el gráfico en el módulo de registro de dolor de la aplicación web.



R06: Gestión de ejercicios terapéuticos.

R06F01: Crear y consultar ejercicios disponibles.

- **R06F01T01:** Utilizar tabla 'exercise'.
 - R06F01T01P01: Inserción de los ejercicios con la URL del vídeo.
- **R06F01T02:** Consultar los ejercicios activos.
 - R06F01T02P01: Obtención mediante peticiones GET a la API.
- **R06F01T03:** Acceder a videos de los ejercicios en la interfaz gráfica.
 - R06F01T03P01: Implementar a través de QWebEngineView().

R07: Asignación de ejercicios a pacientes.

R07F01: Asociar ejercicios mediante planificación terapéutica.

- **R07F01T01:** Uso de tabla 'exercise_assignment'.
 - R07F01T01P01: Registro correcto de frecuencia, series y repeticiones.
- **R07F01T02:** Consulta de ejercicios asignados al paciente.
 - R07F01T02P01: Visualización correcta en la interfaz gráfica de la aplicación de escritorio.
 - R07F01T02P01: Visualización correcta en la interfaz gráfica de la aplicación web.



- **R07F01T03:** Mejora de la experiencia de usuario (UX) mediante selectores dinámicos.
 - R07F01T03P01: Implementación de QComboBox con búsqueda predictiva para evitar errores de inserción de ID manuales.
 - R07F01T03P02: Carga asíncrona desde la API al abrir los diálogos.

R08: Registro de ejercicios realizados.

R08F01: Registrar la ejecución de ejercicios asignados.

- **R08F01T01:** Uso de la tabla *'exercise_done'*.
 - R08F01T01P01: Inserción tras marcar ejercicio realizado.
- **R08F01T02:** Añadir interactividad desde la interfaz gráfica.
 - R08F01T02P01: El usuario podrá marcar el ejercicio realizado a través de un botón en la web.

R09: Arquitectura cliente - servidor.

R09F01: Implementar API REST mediante FastAPI.

- **R09F01T01:** Desarrollo de endpoints CRUD conectados a PostgreSQL.
 - R09F01T01P01: Validación mediante Swagger UI.



R09F02: Interfaz gráfica de escritorio para el fisioterapeuta (PySide6).

- **R09F02T01:** Realizar peticiones HTTP desde cliente.
 - R09F02T01P01: Recepción y visualización correcta de datos.
- **R09F02T02:** Desarrollo a través del sistema MVC de la aplicación de escritorio.
 - R09F02T02P01: Crear la parte gráfica en la carpeta View.
 - R09F02T02P02: Generar la parte lógica en la carpeta Controller.
 - R09F02T02P03: Establecer los objetos en la carpeta Models.
- **R09F02T03:** Estandarización estética mediante hojas de estilo (QSS) en la aplicación de escritorio.
 - R09F02T03P01: Separación de la lógica de negocio y la capa de presentación.
 - R09F02T03P02: Implementación de diseño responsive y jerarquía visual de botones (Menú, Cancel).

R09F03: Interfaz Web para el paciente (React + TypeScript).

- **R09F03T01:** Consumo de la API REST desde el navegador.
 - R09F03T01P01: Gestión de la autenticación mediante JWT (JSON Web Tokens) almacenados de forma segura.
 - R09F03T01P02: Sincronización de datos en tiempo real (Citas, ejercicios asignados).
- **R09F03T02:** Desarrollo del portal web mediante React.
 - R09F03T02P01: Implementación de tipado fuerte con TypeScript para asegurar la integridad de los datos de la API.
 - R09F03T02P02: Crear la parte gráfica en la carpeta Pages.
 - R09F03T02P03: Generar componentes reutilizables en la carpeta Components.
 - R09F03T02P04: Establecer los objetos en la carpeta Models.

- **R09F03T03:** Visualización de recursos multimedia externos.
 - R09F03T03P01: Integración de reproductores embebidos para los vídeos de YouTube.
 - R09F03T03P02: Optimización de la carga de medios para dispositivos móviles.
- **R09F04T04:** Estandarización estética mediante Tailwind CSS en la aplicación web.
 - R09F04T04P01: Separación de la lógica de negocio y la capa de presentación.
 - R09F03T04P02: Creación de componentes reutilizables (Cards, Sliders, Buttons) con Tailwind CSS.

R10: Infraestructura y Despliegue Cloud.

R10F01: Despliegue en Amazon Web Services (AWS).

- **R10F01T01:** Configuración de entorno de ejecución en AWS Elastic Beanstalk.
 - R10F01T01P01: Gestionar variables de entorno para seguridad.
 - R10F01T01P02: Configuración del servidor para producción.
- **R10F02T02:** Implantación de la base de datos en AWS RDS.
 - R10F02T02P01: Configuración de Security Groups para permitir acceso exclusivo desde la IP del backend.
 - R10F02T02P02: Migración de esquemas de datos desde el entorno local a producción.



R11: Automatización y CI/CD (GitHub Actions).

R11F01: Implementar un flujo de trabajo de integración y despliegue continuo.

- **R11F01T01:** Configuración de Workflow en GitHub Actions.
 - R11F01T01P01: Creación del archivo .yml para automatizar las tareas al hacer git push.
 - R11F01T01P02: Configuración del entorno (Runner) con la versión correcta de Python.
- **R11F01T02:** Automatización del Despliegue en AWS.
 - R11F01T02P01: Configuración de GitHub Secrets para almacenar de forma segura las credenciales de AWS (Access Keys).
 - R11F01T02P02: Automatización del empaquetado y envío del código a AWS Elastic Beanstalk tras pasar las validaciones.
- **R11F01T03:** Calidad de Código y Testing.
 - R11F01T03P01: Ejecución automática de pruebas unitarias antes de desplegar.

DESCRIPCIÓN

Arquitectura Cliente - Servidor.

En este proyecto se ha optado por la **arquitectura cliente – servidor**. Esta arquitectura es una de las más populares en la construcción de aplicaciones en la actualidad. Participan principalmente dos componentes: uno es el cliente que utiliza los servicios y otro es el servidor que proporciona estos servicios. Entre ambos se efectúa una comunicación de red, normalmente a través de internet (protocolo HTTP).

Actualmente en internet se utiliza el término backend para la parte de la aplicación que hace de servidor y frontend para la parte de la aplicación que se ejecuta en el cliente. (Zúñiga, Todo sobre la arquitectura cliente-servidor, 2025).

Además, el proyecto se puede segmentar en tres capas principales:

- **Frontend o Capa de Presentación (Cliente):**
 - **Frontend – Desktop o de Escritorio**

Esta parte de la aplicación está enfocada al cliente, en concreto al fisioterapeuta. El desarrollo de esta parte del frontend ha sido focalizado en generar la interfaz gráfica de la aplicación de escritorio, hacer el entorno funcional, lograr una experiencia para el usuario final satisfactoria e intuitiva y hacer una parte visual atractiva para el cliente.

Las tecnologías principales usadas en esta capa han sido **Python + PySide6**. Además, en esta capa se ha usado la librería **Requests**, lo que ha permitido hacer peticiones de creación (POST), obtención (GET), actualización (PATCH) o eliminación (DELETE) a través de peticiones HTTP a mi backend.

Las funciones que cumple son: mostrar la interfaz al fisioterapeuta, capturar datos a través de formularios y comunicarse con la API para enviar y recibir información de la base de datos.



- **Frontend – Web**

Por otro lado, está la sección web de la aplicación, más dirigida al paciente. Esta parte del desarrollo está enfocada en ser mucho más visual y atractiva para el cliente, además de priorizar la accesibilidad y la consistencia visual. Además, se ha realizado un entorno con una navegación fluida y sencilla para hacer la experiencia del paciente lo más satisfactoria posible.

Las tecnologías clave en esta capa son **React.js + TypeScript**, lo que ha permitido lograr una interfaz moderna basada en componentes reutilizables y un tipado fuerte que ha garantizado la robustez del código. Por otro lado, **Tailwind CSS** ha sido el framework utilizado para realizar un diseño limpio y responsive, adaptable a dispositivos móviles y de escritorio. Por último, **Axios** ha sido la librería utilizada para realizar las peticiones a la API de manera asíncrona.

Las funciones que cumple esta parte son: mostrar la interfaz al paciente, capturar los inputs para poder comunicar al fisioterapeuta, presentar una interfaz responsive y adaptativa y mejorar la experiencia del usuario.

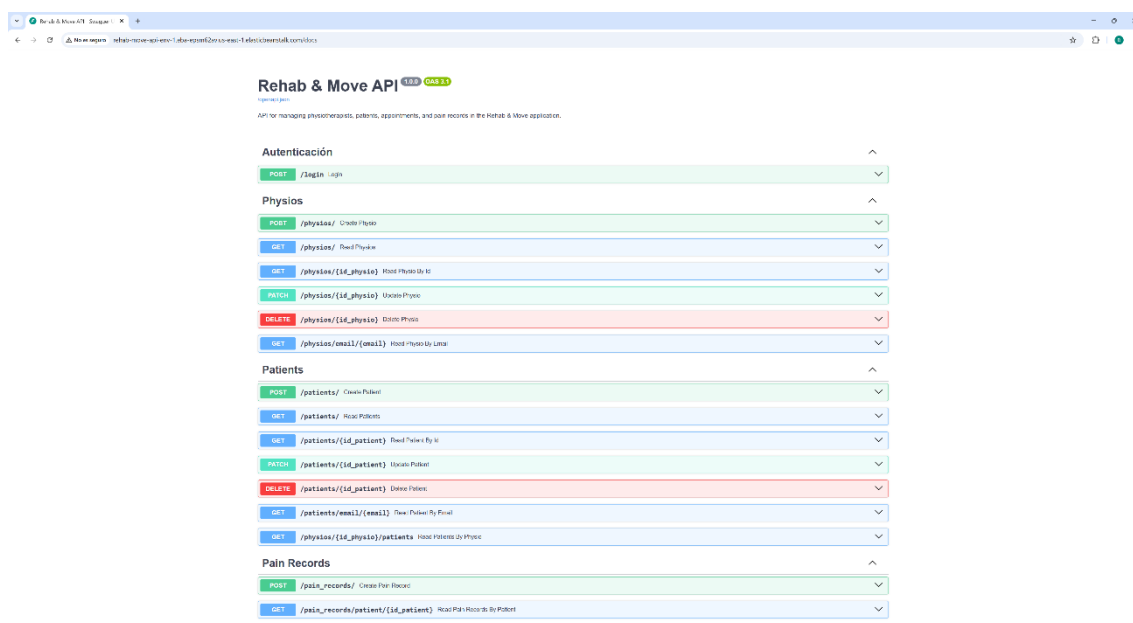
- **Backend o Capa de Negocio (Servidor):**

Esta capa podría considerarse el “cerebro” de la aplicación, siendo esta la parte invisible de Rehab&Move.

La tecnología principal de esta capa ha sido **Python + FastAPI**, además de librerías como **Pydantic** para la validación de datos, **SQLAlchemy** para traducir mis entidades de PostgreSQL a clases del código python o **Passlib** para la gestión de contraseñas.

El backend se encarga de procesar las solicitudes del frontend, interactuar con los datos almacenados en mi base de datos que a su vez se encuentra en AWS RDS y por último devolver la información estructurada en formato JSON.

Por otro lado, he utilizado **AWS Elastic Beanstalk** como servidor en la nube para desplegar mi parte backend de la aplicación y así, poder realizar consultas a la base de datos desde cualquier lugar y descentralizar la información de mis dispositivos locales.



Captura de pantalla del Swagger UI de Rehab&Move API en el servidor web.

- **Capa de Datos o Persistencia (Infraestructura Cloud):**

Como he adelantado en la Capa de Negocio o backend este proyecto ha optado por los servicios gestionados en la nube de Amazon Web Services (AWS).

He utilizado la tecnología de **AWS RDS** para poder almacenar la información estructurada de la aplicación a través de PostgreSQL (fisioterapeutas, pacientes, citas, registros de dolor...). La comunicación se realiza mediante el ORM de SQLAlchemy, disminuyendo la complejidad de consultas SQL manuales.

Además, al tener la base de datos en un servidor web, permite descentralizar la información, garantizando que cualquier usuario pueda acceder a los datos clínicos desde cualquier dispositivo.

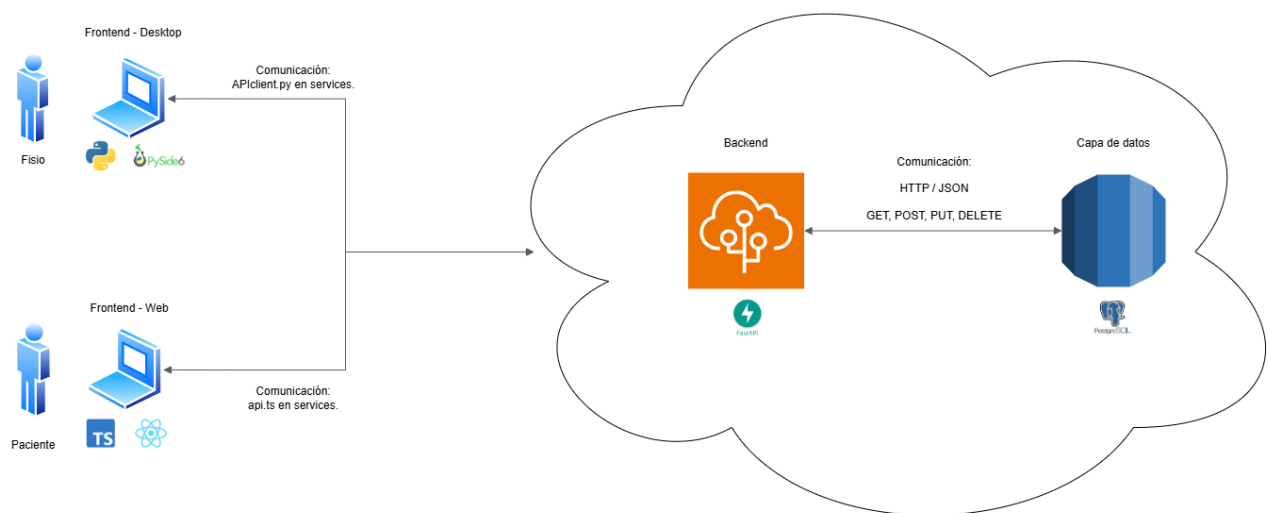


Diagrama de arquitectura cliente-servidor.

Caso de Uso: Asignar un ejercicio a un paciente.

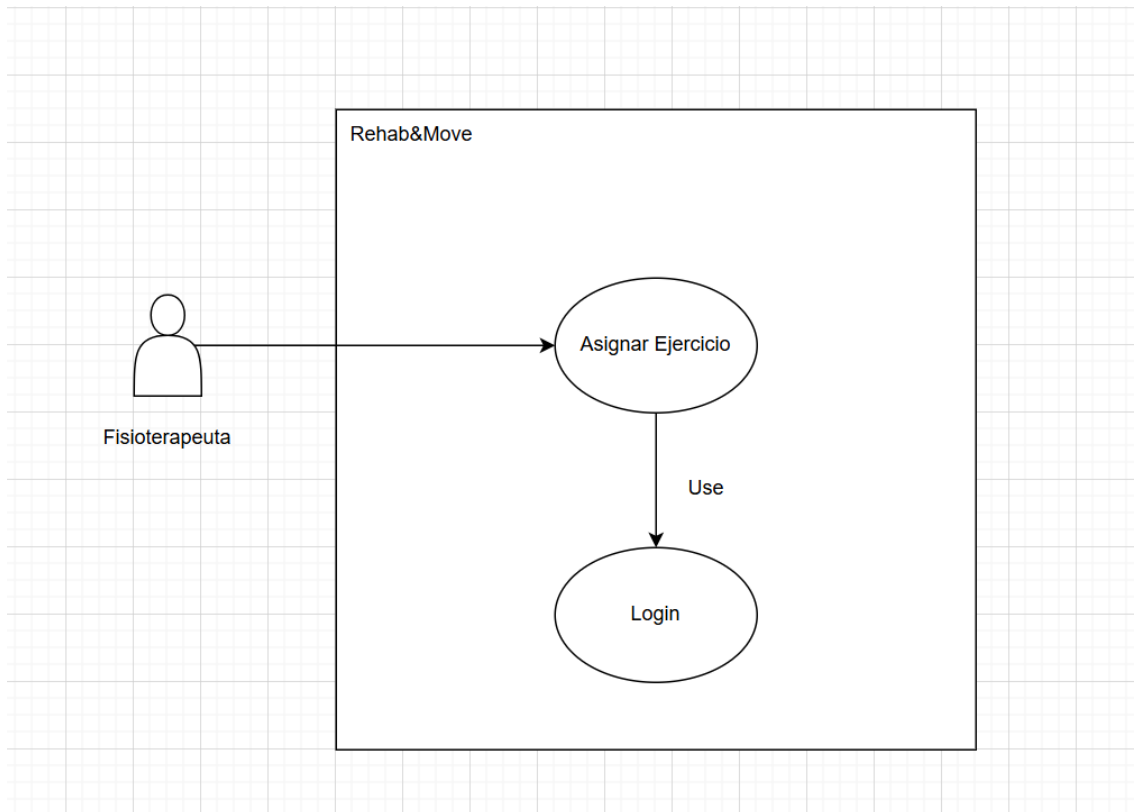


Ilustración: Caso asignar un ejercicio a un paciente.



DESCRIPCIÓN: El fisioterapeuta selecciona un paciente y un ejercicio para crear un plan de entrenamiento.

PRECONDICIONES:

Fisioterapeuta logueado en el sistema.
Paciente y ejercicio dado de alta en el sistema.

POSTCONDICIONES:

Nueva asignación registrada en la BD.
Aparición del ejercicio asignado en el panel de ejercicios asignados.
El paciente puede visualizar su ejercicio en el panel.

DATOS ENTRADA

- ID Paciente.
- ID Ejercicio.
- Frecuencia semanal.
- Series.
- Repeticiones.
- Fecha de inicio.
- Fecha fin.

DATOS SALIDA

- Ventana emergente que confirma el registro con éxito.

TABLAS:

- Patient
- Exercise
- Exercise_assignment

CLASES:

- AddExerciseAssigDialog.py
- ApiClient.py
- ExerciseAssignment (Pydantic)

INTERFACES:

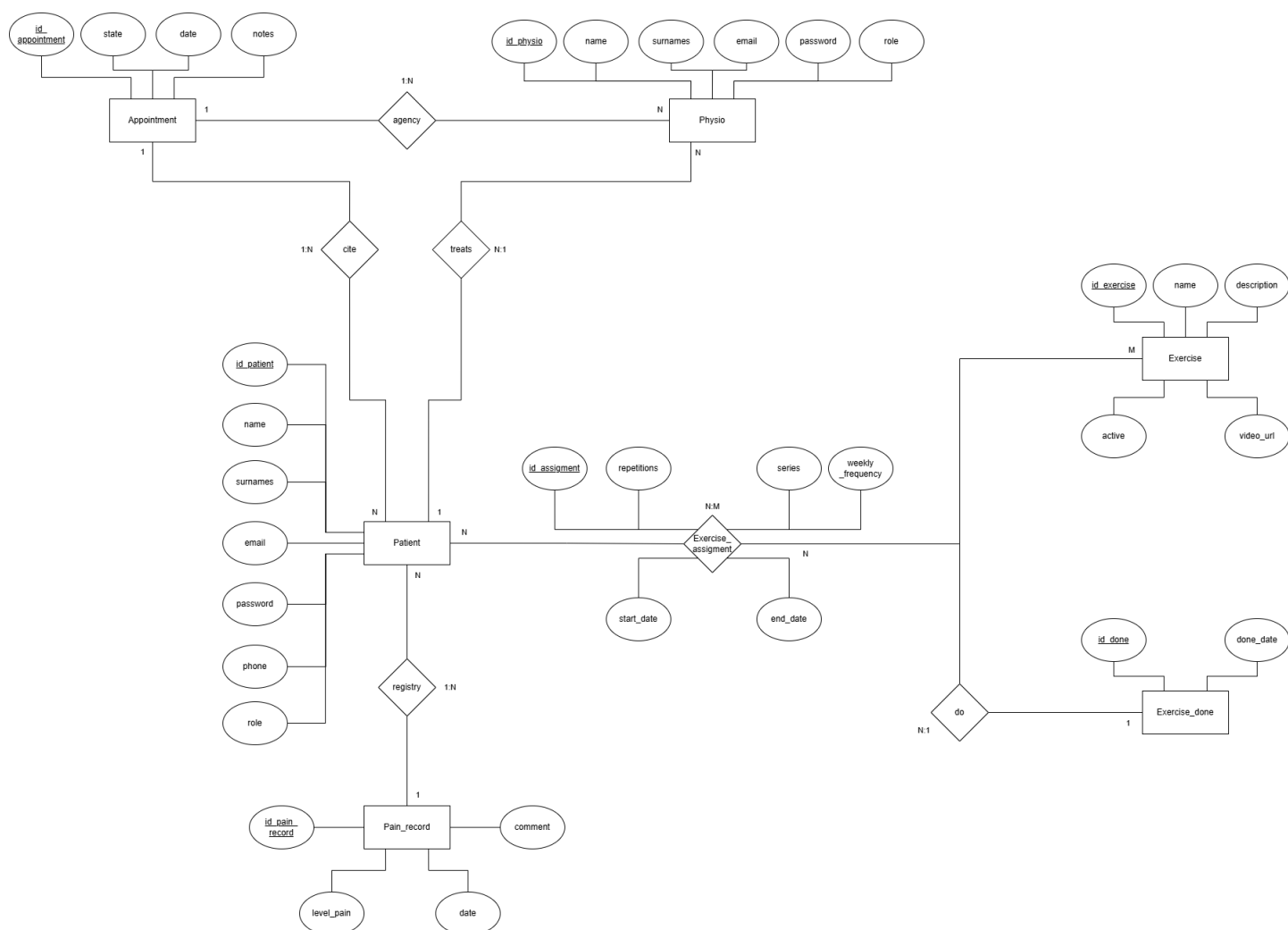
- add_exercise_assig_dialog.py
- exercise_dashboard.py
- exercise_assigned_dashboard.py

DISEÑOS

En este apartado, voy a mostrar los diseños realizados para este proyecto, desde los diagramas hasta las interfaces desarrolladas:

Diagrama Entidad - Relación.

Un diagrama entidad – relación es una herramienta gráfica que representa la estructura de una base de datos relacional, mostrando como interactúan las entidades entre sí. Utiliza símbolos rectángulos para entidades, rombos para relaciones y óvalos para atributos, lo que permite visualizar la lógica de los datos. (Lucidchart, s.f.).





Entidad 'Physio'.

Representa a cada fisioterapeuta registrado en la aplicación.

Relaciones:

- Entidad 'Patient' (1:N): Un fisioterapeuta puede tratar a varios pacientes, pero un paciente está asignado únicamente a un fisioterapeuta.
- Entidad 'Appointment' (1:N): Un fisioterapeuta asigna varias citas, pero una cita está relacionada únicamente con un fisioterapeuta.

Entidad 'Patient'.

Refleja a los pacientes registrados.

Relaciones:

- Entidad 'Physio' (N:1): Un paciente es tratado por un fisioterapeuta, pero un fisioterapeuta trata a varios pacientes.
- Entidad 'Appointment' (1:N): Una cita está asignada a un paciente, pero un paciente tiene varias citas asignadas.
- Entidad 'Pain record' (1:N): Un paciente registra varias veces su nivel de dolor, pero un registro es hecho únicamente por un paciente.
- Entidad 'Exercise' (N:M): Un paciente tiene varios ejercicios asignados y, además, un ejercicio puede ser realizado por varios pacientes. Esta relación conlleva la creación de la tabla intermedia 'Exercise_assignment'.

Entidad 'Appointment'.

Muestra las citas asignadas por el fisio a cada paciente.

Relaciones:

- Entidad 'Physio' (N:1): Cada cita está realizada por un fisioterapeuta, pero un fisioterapeuta genera varias citas.
- Entidad 'Patient' (N:1): Cada cita está asignada a un paciente, sin embargo, un paciente tiene agenciadas varias citas.



Entidad 'Pain_record'.

Relacionada con los registros de dolor hechos por cada paciente.

Relaciones:

- Entidad 'Patient' (N:1): Un registro está hecho por un paciente, pero un paciente realiza varios registros de dolor.

Entidad 'Exercise'.

Registra los ejercicios que están disponibles en la aplicación.

Relaciones:

- Entidad 'Patient' (N:M): Un ejercicio puede ser realizado por varios pacientes, además, un paciente ejecuta varios ejercicios. Esta relación genera la tabla intermedia 'Exercise_assignment'.

Entidad 'Exercise_done'.

Refleja los ejercicios hechos por el paciente.

Relaciones:

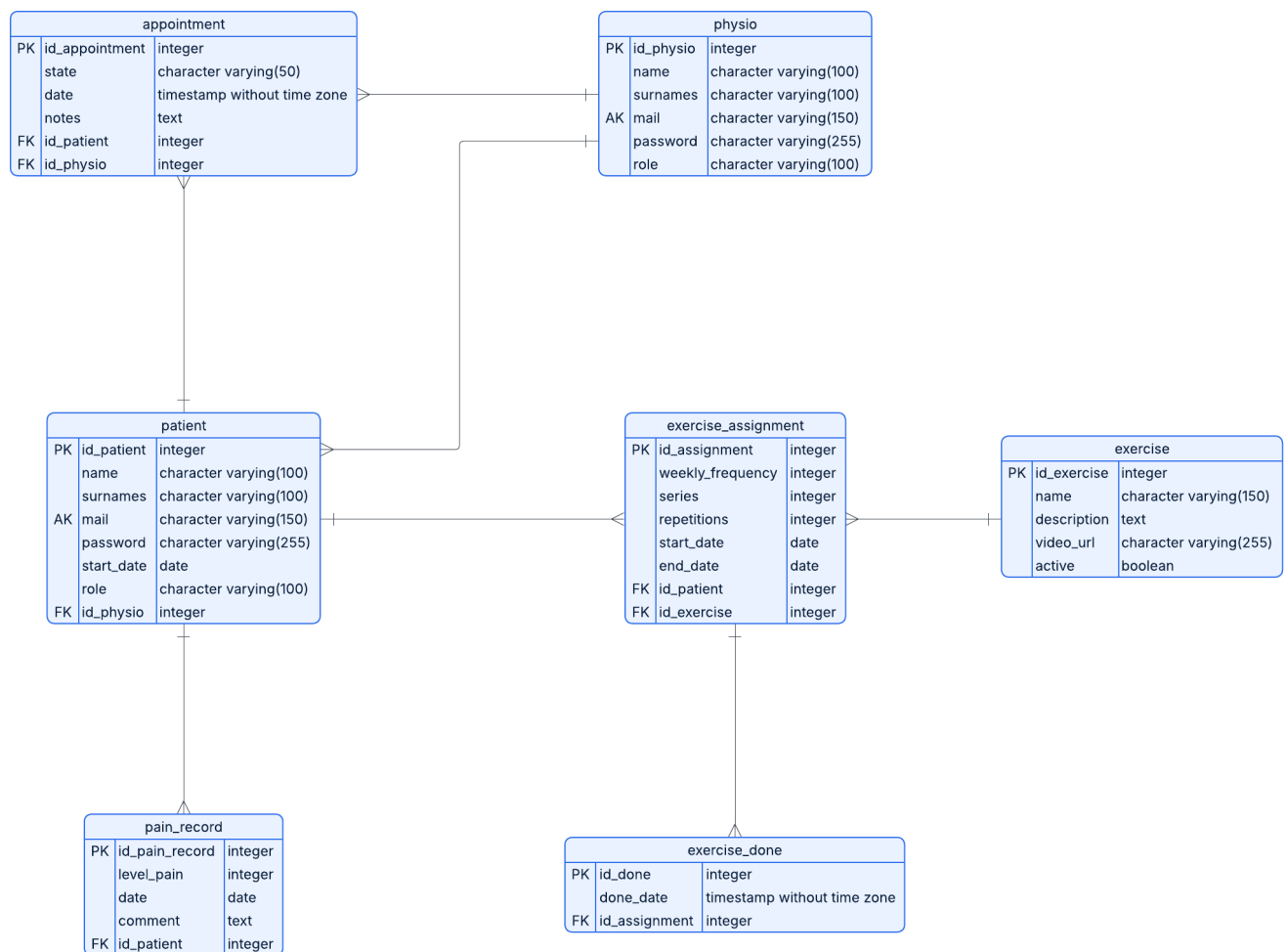
- Entidad intermedia 'Exercise_assignment' (1:N): Un ejercicio asignado a un paciente puede tener muchos registros de realización del ejercicio, mientras que un registro de realización está únicamente relacionado con un ejercicio asignado.

Entidad intermedia 'Exercise_assignment'.

La tabla central, es una tabla intermedia que plasma los ejercicios asignados a cada paciente, por lo que establece la relación N:M entre la tabla 'Patient' y la tabla 'Exercise'. Además, esta tabla intermedia tiene relación directa con la tabla 'Exercise_done' que representa el registro de ejercicios realizados.

Diagrama de la base de datos.

Un diagrama de base de datos es una herramienta visual que ayuda a presentar la base de datos mediante tablas, columnas, claves y relaciones. Con este diagrama se puede visualizar el diseño completo de la base de datos, lo que facilita la comprensión del desarrollo de esta. (Wondershare, 2025).





Logo.



Logo desarrollado a través de inteligencia artificial, en concreto Gemini, esta puede generar imágenes de uso libre para trabajos académicos o proyectos personales. La figura representa una persona a través de una imagen dinámica, simbolizando la metamorfosis del proceso de recuperación. Además, incluye en la parte inferior el nombre de la aplicación, dándole un diseño más personalizado. El logo es acompañado de colores pastel, ya que busca transmitir calma, confianza y salud. (Gemini, s.f.)

Interfaces de la aplicación: Aplicación de escritorio.

Login.

Rehab & Move - Login

¡Bienvenido!

Correo electrónico

Contraseña

Entrar



Lista de Pacientes					
ID	Nombre	Apellidos	Email	Teléfono	Fecha Inicio
1	Luis	Lopez de la Cruz	luislopez@correo.com	612345678	2023-03-15
2	Isabel	Mora Hernandez	isabelmora@gmail.com	698765432	2023-04-10
3	Diego	Rivera Martinez	diego.rivera@outlook.com	777777777	2023-05-18
4	María	Perez Lopez	maria.perez@correo.com	654321098	2023-06-22
5	Carlos	García Gomez	carlos.garcia@outlook.com	555555555	2023-07-05
6	Ana	Sanchez Ruiz	ana.sanchez@correo.com	444444444	2023-08-12
7	Roberto	Alvarez Torres	roberto.alvarez@gmail.com	333333333	2023-09-01
8	Patricia	Jimenez Diaz	patricia.jimenez@outlook.com	222222222	2023-10-15
9	Diego	Castro Ortiz	diego.castro@correo.com	111111111	2023-11-20
10	Sofía	Perera Velez	sofia.perera@outlook.com	999999999	2024-01-05
11	Lucas	Molina Lopez	lucas.molina@correo.com	888888888	2024-02-18
12	Valeria	Costa Ruiz	valeria.costa@gmail.com	777777777	2024-03-25

Añadir Nuevo Paciente

Datos del Paciente

Nombre:

Apellidos:

Email:

Teléfono:

Contraseña:

Cancelar

Guardar Paciente

Modificar Paciente

Datos del Paciente

Nombre:

Roberto

Apellidos:

Cano Lara

Email:

roberto@email.com

Teléfono:

666777888

Contraseña:

Contraseña segura

Cancelar

Guardar Paciente

Calendario de sesiones (fisioterapeuta).

Calendario de sesiones

	14	15	16	17	18	19	20
18	27	28	29	30	1	2	3
19	4	5	6	7	8	9	10
20	11	12	13	14	15	16	17
21	18	19	20	21	22	23	24
22	25	26	27	28	29	30	31
23	1	2	3	4	5	6	7

Ventanas emergentes de visualizar sesiones diarias y añadir citas.

Horas disponibles - 12/05/2026

Gestión de citas para el día: 12/05/2026

- 09:00 - OCUPADO (Lucía Luque de la Casa)
- 10:00 - OCUPADO (Enrique Maya Fernández)
- 11:00 - OCUPADO (Iván Ríoja Marrero)
- 12:00 - OCUPADO (Manuel Prados López)
- 13:00 - OCUPADO (Elena García Sanz)
- 14:00 - LIBRE
- 15:00 - LIBRE
- 16:00 - LIBRE
- 17:00 - LIBRE

Agregar Cita

Nueva Cita

Paciente:

Notas:

Guardar Cita Cancelar

Agregar Cita

Nueva Cita

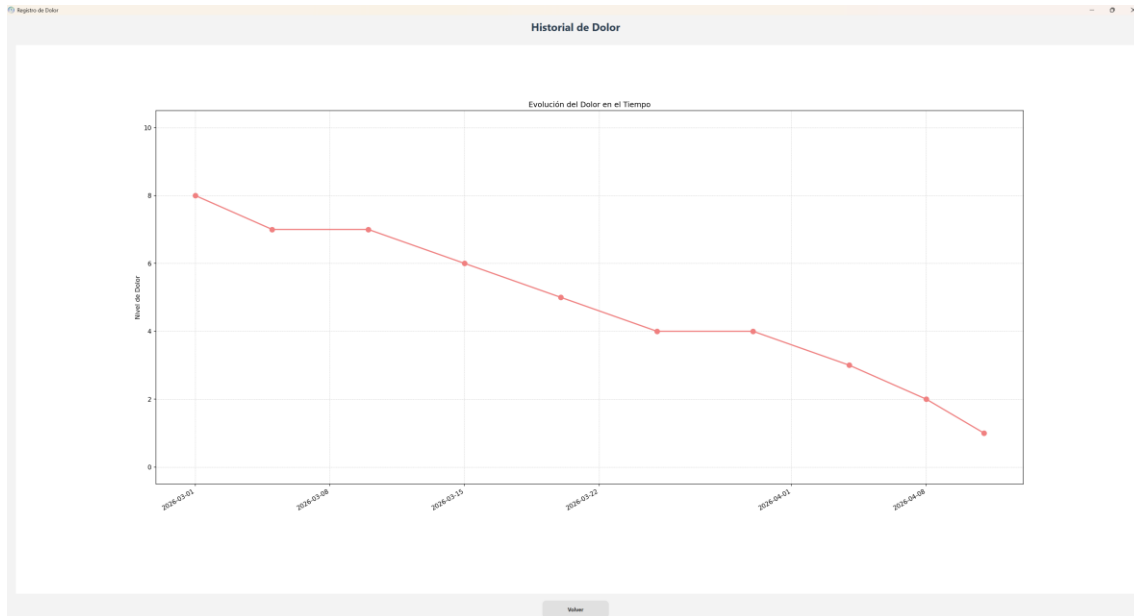
Paciente:

Notas:

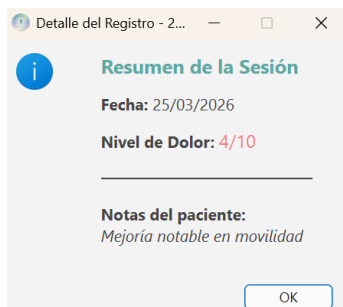
Guardar

- 1 - Lucía Luque de la Casa
- 2 - Enrique Maya Fernández
- 3 - Iván Ríoja Marrero
- 4 - Marta García Fernández
- 5 - Johan Gutierrez del Pino
- 10 - Manuel Prados López
- 11 - Elena García Sanz

Registro de dolor de un paciente.



Ventana emergente con detalle de un registro.



Ventana emergente que muestra el detalle de un registro de dolor. Incluye un ícono de información, el título 'Resumen de la Sesión', la fecha, el nivel de dolor y las notas del paciente.

Resumen de la Sesión

Fecha: 25/03/2026

Nivel de Dolor: 4/10

Notas del paciente:
Mejoría notable en movilidad

OK

Menú de ejercicios.

Gestión de Ejercicios

ID	Nombre	Descripción	URL	Activo
3	Sentadilla Asistida	Ejercicio para fortalecer cuádriceps y glúteos apoyándose en una silla o pared para...	https://www.youtube.com/watch?v=7f6U3wC1	✓
4	Plancha Abdominal	Fortalecimiento del core manteniendo una línea recta desde los hombros hasta los...	https://www.youtube.com/watch?v=4a6g3d3d2G	✓
5	Puente de Glúteos	Elevación de pelvis tumbado boca arriba para trabajar glúteos y zona lumbar.	https://www.youtube.com/watch?v=vaF8t3uUd0	✓
6	Punto Pájaros (Bird Dog)	Estabilidad lumbar extendiendo brazo y pierna contraria simultáneamente en...	https://www.youtube.com/watch?v=C1H9Wah7mQ8	✓
7	Gato-Camello	Movilidad de la columna vertebral alternando entre arquear y curvar la espalda.	https://www.youtube.com/watch?v=ZC_3p3LA68	✓
8	Retración Escapular	Juntar los omóplatos hacia atrás para mejorar la postura y fortalecer el trapecio medio.	https://www.youtube.com/watch?v=V4a0H4H9mP	✓
9	Equilibrio a una Pierna	Mejora de la propiocepción y estabilidad del talón manteniendo el equilibrio 30...	https://www.youtube.com/watch?v=1T8W4P_Z_Rnq	✓
10	Dead Bug	Ejercicio de control motor abdominal manteniendo la espalda baja pegada al suelo.	https://www.youtube.com/watch?v=vd8J8GvPC8	✓
11	Estiramiento Piramidal	Alivio de la tensión en el glúteo y nervio ciático cruzando una pierna sobre la otra.	https://www.youtube.com/watch?v=C123gVQ2h	✓
12	Elevación de Talones	Fortalecimiento de gemelos y talón elevándose sobre los puntos de los pies.	https://www.youtube.com/watch?v=5dR9h705vK	✓
13	Rotación Extrema de Hombros	Uso de banda elástica para fortalecer el manguito rotador manteniendo el codo...	https://www.youtube.com/watch?v=5dR9h705vK	✓
14	Superman Invertido	Fortalecimiento de la cadena posterior elevando tronco y piernas boca abajo.	https://www.youtube.com/watch?v=MSQyH5Cse	✓
15	Estiramiento de Isquiotibiales	Estiramiento de la parte posterior del muslo con la ayuda de una toalla o banda.	https://www.youtube.com/watch?v=C123gVQ2h	✓
16	Inclinación Pélvica	Movimiento suave de la pelvis para aliviar el dolor lumbar y activar el transverso.	https://www.youtube.com/watch?v=Vd8h3M4W8	✓
17	Abducción de Cadera Lateral	Fortalecimiento del glúteo medio elevando la pierna tumbado de lado.	https://www.youtube.com/watch?v=u2p6a7WED4	✓
18	Flexiones en Pared	Versión suave de flexiones para trabajar pectorales y tríceps sin cargar demasiado e...	https://www.youtube.com/watch?v=Jd8R3v4M	✓
19	Movilidad Cervical	Rotaciones suaves del cuello para reducir la tensión en el trapecio superior.	https://www.youtube.com/watch?v=Vd8h3M4W8	✓
20	Estiramiento de Pectorales	Apertura de pecho apoyando el antebrazo en el marco de una puerta.	https://www.youtube.com/watch?v=5dR9h705vK	✓
21	Isométrico de Cúbito	Apretar el mudo contra la cama con una toalla bajo la rodilla para rehabilitación...	https://www.youtube.com/watch?v=dg4kgd3Tpe	✓
22	Rotación de Tronco Sentado	Mejora de la movilidad torácica girando el cuerpo suavemente hacia ambos lados.	https://www.youtube.com/watch?v=5dR9h705vK	✗

Ventana emergente con información del ejercicio.

Video del ejercicio

PUENTE DE GLÚTEO

Elevación de pelvis tumbado boca arriba para trabajar glúteos y zona lumbar.

YouTube ES

Buscar

Iniciar sesión

TO DO A

GLUTE BRIDGE

0:00 / 2:08

Cómo hacer un puente de glúteos | La forma correcta | Well+Good

Doblado automáticamente

Well+Good
754 K suscriptores

Suscribirse

23 K

Compartir

Cerrar



Ventanas emergentes de añadir, modificar y asignar un ejercicio.

Añadir Nuevo Ejercicio

Datos del Ejercicio

Nombre:

Descripción:

URL del video:

Modificar Ejercicio

Datos del Ejercicio

Nombre:

Descripción:

URL del video:

Activo: ☒

Asignar Ejercicio

Paciente:

Ejercicio:

Frecuencia semanal:

Número de series:

Número de repeticiones:

Fecha de inicio:

Fecha de fin:

Menú de ejercicios asignados.

Ejercicios Asignados - Paciente 2

ID	Frecuencia Semanal	Series	Repeticiones	Fecha Inicio	Fecha Fin	Ejercicio	Estado
8	5	3	10	2026-04-25	2026-05-02	Superman Invertido	✗
9	5	2	10	2026-04-25	2026-05-02	Dead Bug	✗
10	5	5	5	2026-04-25	2026-05-02	Modificado Cervical	✗
11	7	5	10	2026-04-25	2026-05-02	Ejercicio Prensión	✗

Ventanas emergentes de asignar ejercicio y editar plan de ejercicio.

Asignar Ejercicio

Paciente:

Ejercicio:

Frecuencia semanal:

Número de series:

Número de repeticiones:

Fecha de inicio:

Fecha de fin:

Editar Plan de Ejercicio

Paciente:

Ejercicio:

Frecuencia semanal:

Número de series:

Número de repeticiones:

Fecha de inicio:

Fecha de fin:

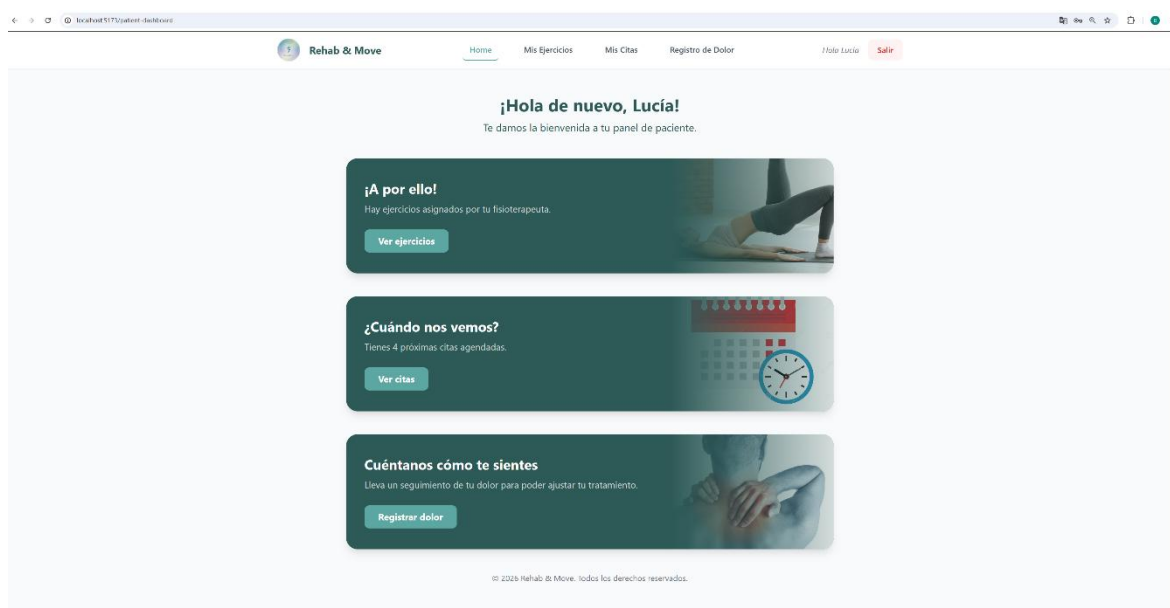


Interfaces de la aplicación: Aplicación web.

Login.

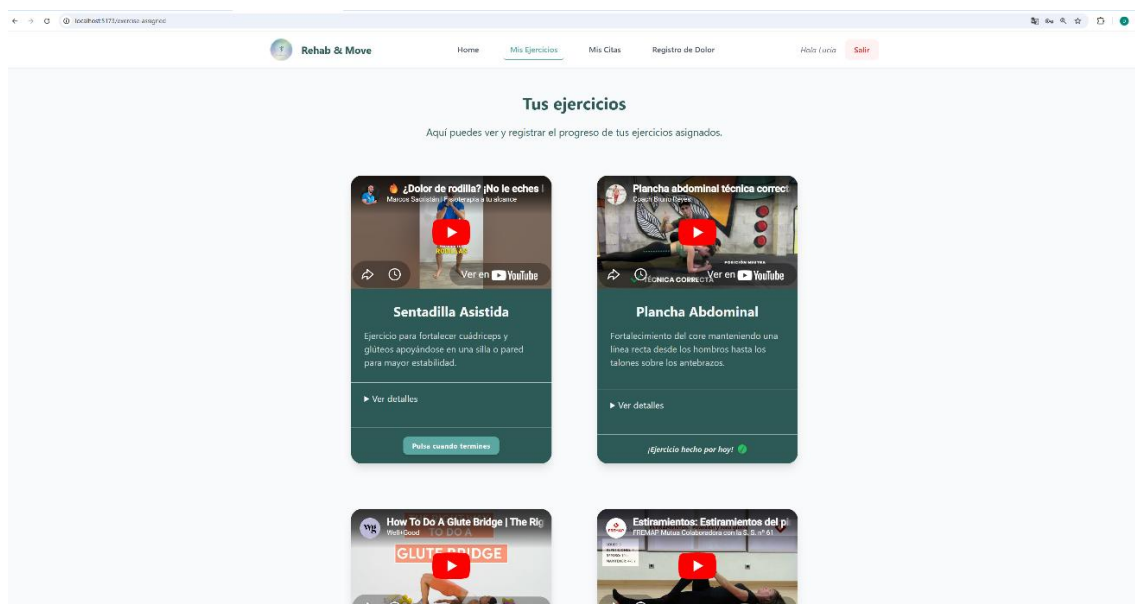
The login interface is a white card centered on a light gray background. At the top is the Rehab & Move logo. Below it, the text "¡Bienvenido!" is displayed in a teal color. There are two input fields: "Email" with the placeholder "correo@ejemplo.com" and "Contraseña" with a masked password "*****". At the bottom are two buttons: a teal "Entrar" button and a gray "Cancelar" button.

Dashboard del paciente.

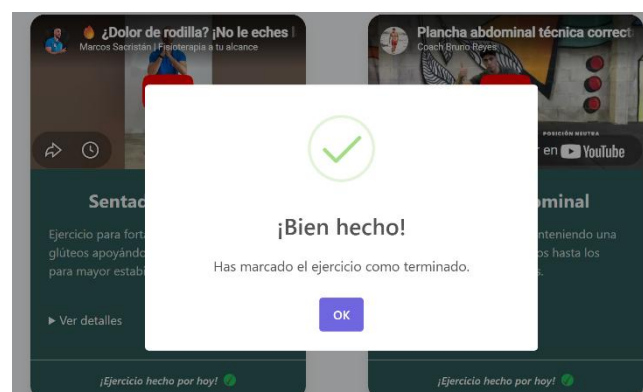
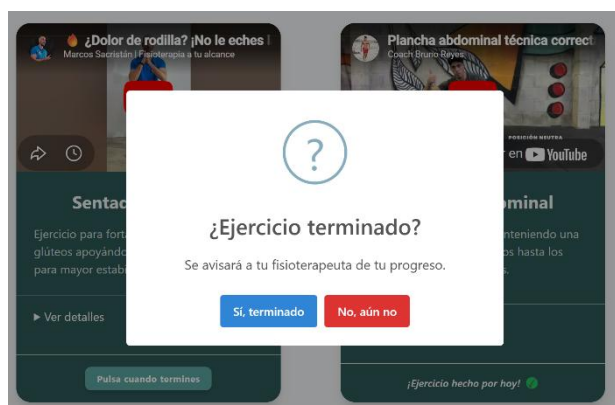




Página Mis Ejercicios.



Ventanas emergentes en Mis Ejercicios.





Página Mis Citas.

The screenshot shows the 'Mis Citas' page of the Rehab & Move application. The page has a navigation bar with 'Home', 'Mis Ejercicios', 'Mis Citas' (active), 'Registro de Dolor', 'Hola Lucía', and 'Salir'. The main heading is 'Tus citas' with the subtitle 'Aquí puedes ver y gestionar tus citas con tu fisioterapeuta.' Below this, there are two main sections: 'Mis Citas' and 'miércoles, 20 de mayo de 2026'. The 'Mis Citas' section contains a calendar for May 2026, with the 20th highlighted in green. The 'miércoles, 20 de mayo de 2026' section shows details for a session with 'Fisioterapeuta: David Prados Medina' at 'Hora: 16:00' for 'Consejería: Tratamiento clínico'.

Página Registro de Dolor.

The screenshot shows the 'Registro de dolor' page of the Rehab & Move application. The page has a navigation bar with 'Home', 'Mis Ejercicios', 'Mis Citas', 'Registro de Dolor' (active), 'Hola Lucía', and 'Salir'. The main heading is 'Registro de dolor' with the subtitle 'Aquí puedes registrar tu nivel de dolor para que tu fisioterapeuta pueda monitorear tu progreso.' Below this, there is a form titled '¿Cómo te sientes hoy?' with a large '5' and 'ESCALA DE DOLOR'. A slider below the number shows 'SIN DOLOR', 'MODERADO', and 'MÁXIMO'. There is a text input field for '¿Algún comentario para tu fisio?' with the example text 'Ej: Me ha molestado más cuando llevaba un rato sentado...'. At the bottom of the form is a button labeled 'Registrar dolor diario'. The footer of the page reads '© 2026 Rehab & Move. Todos los derechos reservados.'

Ventanas emergentes de Registro de Dolor.



¿Registrar nivel de dolor?

Se guardará tu nivel de dolor para que tu fisioterapeuta pueda revisarlo.

Sí, registrarNo, cancelar



¡Registrado!

Tu nivel de dolor ha sido registrado.

OK

Página Registro de Dolor (Registro hecho).

Rehab & Move

HomeMis EjerciciosMis CitasRegistro de DolorHalo EnriqueSalir

Registro de dolor

Aquí puedes registrar tu nivel de dolor para que tu fisioterapeuta pueda monitorear tu progreso.

Registro realizado:

Nivel de dolor: 2

Comentario: He mejorado mucho esta semana

Mañana podrás realizar un nuevo registro.

© 2026 Rehab & Move. Todos los derechos reservados.

TECNOLOGÍA

Las tecnologías utilizadas para este proyecto son:



PostgreSQL.

Sistema de gestión de base de datos relacionales orientada a objetos de código abierto. (Microsoft, s.f.)

Se ha usado en este proyecto como gestor de la base de datos debido a su fiabilidad, escalabilidad y rendimiento.



PgAdmin4.

Herramienta de administración gráfica de código abierto para PostgreSQL. (Morales, s.f.)

Su función en este proyecto ha estado enfocada en la administración de la base de datos gracias a su interfaz gráfica intuitiva.



Python.

Lenguaje de programación de alto nivel, interpretado, de código abierto y multiparadigma. (Oracle, s.f.)

Lenguaje de programación principal usado en el proyecto, seleccionado debido a su potencial back-end y su desarrollo de interfaces gráficas.



FastAPI

FastAPI.

Framework moderno y de alto rendimiento enfocado a construir APIs con Python 3.7+, destacado por su velocidad y su uso de tipado de Python. (Tiangolo, s.f.)

Este framework se ha usado en el proyecto para construir la API REST ofreciendo un desarrollo rápido y ágil, además de generar un sistema backend consolidado.



SQLAlchemy.

Es el kit de herramientas SQL y ORM más popular para Python. Permite interactuar con la base de datos relacional utilizando clases y objetos de Python en lugar de escribir consultas SQL manuales. (4Geeks, 2026).

En este proyecto se ha usado esta librería con el objetivo de traducir mis entidades de posgreSQL a código python.



Pydantic.

Biblioteca de Python para validar los datos y la gestión de configuraciones. (Tuychiev, 2025).

Pydantic se ha usado para validar las estructuras de mis modelos generado con SQLAlchemy y así, realizar un control de errores a la hora de enviar mi información a la base de datos.



Uvicorn.

Servidor web que implementa el estándar ASGI y facilita la creación de endpoints en tiempo real. (Q2BStudio, 2026).

Uvicorn se ha usado como motor que permite que mi código Python reciba las peticiones de internet.



Gunicorn.

Es un servidor HTTP WSGI robusto y ligero para Python, diseñado para entornos de producción. (Gunicorn, s.f.).

Servidor utilizado junto a Uvicorn como administrador de procesos. Este trabajo combinado entre Gunicorn y Uvicron permite tener una API robusta, estable y capaz de manejar múltiples usuarios.



Dotenv.

Es un módulo que permite cargar variables de configuración desde un archivo .env en el entorno de ejecución del sistema. (Platzi, 2020).

En este proyecto se ha usado dotenv para mejorar la seguridad, ya que ha permitido separar las contraseñas del código fuente mediante el archivo .env.



Passlib.

Passlib es una biblioteca de seguridad para python diseñada para gestionar el hashing y la verificación de contraseñas. (Collins, 2020).

Esta biblioteca ha sido utilizada para poder manejar las contraseñas de forma segura a través de bcrypt.



PyTest.

Framework de pruebas (testing) de código abierto para Python. (Pykes, 2024)

Usado en este proyecto para realizar test de prueba en el flujo de CI/CD.

SwaggerUI.



Herramienta de código abierto que genera una interfaz web. Permite visualizar, documentar y probar APIs REST directamente desde el navegador. (Salgado, 2024).

En este proyecto se ha utilizado como interfaz interactiva generando documentación autodocumentada bajo el estándar OpenAPI.

PySide6.



Framework de código abierto utilizado para desarrollar interfaces gráficas de usuario (GUI) multiplataforma a través de Python 3.6+. (Python GUIs, 2025)

Librería utilizada en este proyecto para desarrollar la parte gráfica, siendo una opción robusta y potente.

Matplotlib.



Biblioteca de visualización de datos de código abierto para Python, diseñada para crear gráficos estáticos, animados e interactivos de alta calidad en 2D y 3D. (KeepCoding, 2024)

En este proyecto se utiliza para poder visualizar en la aplicación la gráfica de evolución del dolor del paciente.

Visual Studio Code.



Editor de código ligero pero potente, desarrollado por Microsoft, gratuito y de código abierto. (Zúñiga, ¿Qué es Visual Studio Code y cuáles son sus ventajas?, 2025)

Este editor ha sido seleccionado para el proyecto debido a su fácil manejabilidad y su potencia para desarrollar código.



AWS RDS.

Servicio gestionado de AWS que simplifica la configuración, operación y escalado de base de datos relacionales en la nube. (AWS., Amazon Relational Database Service, 2026).

Se ha utilizado este servicio para almacenar la base de datos *rehab-db* en la nube.



AWS Elastic Beanstalk.

Servicio de AWS que permite a los clientes migrar, implementar y escalar fácilmente sus aplicaciones. (AWS., AWS Elastic Beanstalk, 2026.).

Servicio de la nube dónde se ha subido la API de esta aplicación.



IAM

AWS IAM.

Tecnología de AWS que permite gestionar de forma segura los servicios cloud de Amazon. (AWS., ¿Por qué utilizar IAM?, 2026).

En este proyecto, se ha utilizado IAM para generar usuarios con permisos a los servicios de AWS.



GitHub.

Plataforma en la nube utilizada para alojar, gestionar y colaborar proyectos de código fuente utilizando el sistema de control de versiones git. (GitHub, s.f.).

En este proyecto se ha utilizado GitHub como repositorio web para el código fuente.



Git.

Git es un sistema de control de versiones que realiza un seguimiento de los cambios en los archivos. (GitHub, s.f.).

Tecnología utilizada en el proyecto para realizar el control de versiones.



GitHub Actions

GitHub Actions.

GitHub Actions es una plataforma de integración y despliegue continuos (IC/DC) que te permite automatizar tu mapa de compilación, pruebas y despliegue. (GitHub., s.f.)

Se ha utilizado GitHub Actions para automatizar despliegues a AWS a través de GitHub.



Nginx.

NGINX es un servidor web de código abierto, proxy inverso, balanceador de carga y caché HTTP de alto rendimiento. (Jackson, 2025).

Servidor web que utiliza AWS Elastic Beanstalk por defecto, en este proyecto gestiona el tráfico entre la API e internet.



Python Requests.

Biblioteca de terceros para Python, sencilla y potente, diseñada para realizar peticiones HTTP (GET, POST, etc.) a servidores web. (Gómez, s.f.)

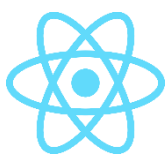
Utilizado en esta app para conectar el backend con el frontend a través de la clase APIClient.



TypeScript.

Lenguaje basado en JavaScript de código abierto creado por Microsoft para el desarrollo web. (Zúñiga, Arsys, 2024).

Se ha usado este lenguaje de programación para la parte frontend-web del proyecto, permitiendo un tipado fuerte lo que conlleva un código más robusto.



React.

Biblioteca de JavaScript de código abierto, creada por Meta y diseñada para construir interfaces de usuario interactivas y escalables de manera rápida y eficiente. (Meta, s.f.).

Esta biblioteca se ha usado en el proyecto para el desarrollo de la parte web, realizando una web responsive y compuesta por componentes reutilizables.



Tailwind CSS.

Framework CSS de bajo nivel, diseñado para agilizar el desarrollo web al permitir la creación de interfaces personalizadas directamente en el HTML. (Huet, 2022).

Framework utilizado para la parte del CSS, lo que ha permitido generar una web moderna y atractiva para el cliente.



Axios.

Biblioteca de JavaScript basada en promesas que se utiliza para realizar solicitudes HTTP desde el navegador o Node.js. (Axios, s.f.).

Biblioteca usada para realizar la comunicación entre la web y la API del proyecto.

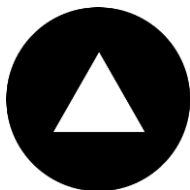
Vite.



Vite es una herramienta de construcción y compilación web que tiene como objetivo proporcionar una experiencia de desarrollo rápida y ágil para proyectos modernos. (Vite, s.f.).

Utilizado en este proyecto para el desarrollo web, permitiendo un desarrollo fácil y rápido.

Vercel.



Plataforma unificada en la nube que permite a los programadores desplegar, gestionar y escalar sus aplicaciones y sitios web. (aplyca, 2026).

En este proyecto se ha utilizado Vercel como herramienta para desplegar la web en producción.

METODOLOGÍA

Para el desarrollo de Rehab&Move, he elegido la **metodología ágil con el marco de trabajo Scrum**. Este es un marco ágil que divide el trabajo en Sprints planificados y que se basa en la definición de roles, artefactos y ceremonias para lograr una entrega iterativa. (Drumond, s.f.).

En mi caso he adaptado los roles actuando como Product Owner y Developer de forma simultánea. Además, esta metodología me ha permitido organizar las tareas en Sprints de una semana.

Como puntos fuertes de esta metodología destacan:

- **Desarrollo incremental:** Me ha permitido trabajar primero en el backend (la base del proyecto) para luego poder desarrollar las interfaces de usuario más cómodamente.
- **Flexibilidad:** Este proyecto está compuesto por varias herramientas tecnológicas (PySide6, FastAPI, React...), la metodología ágil me ha permitido integrar todas las capas sin bloquear el avance global.
- **Integración Continua (CI):** Desde el principio del desarrollo, el uso de Git como control de versiones me ha permitido validar cualquier cambio antes de subirlo a AWS gracias al pipeline de GitHub Actions.



Planificación temporal inicial.

Antes de comenzar con el desarrollo, realicé una estimación previa a la implementación del proyecto, estipulando más o menos 300 horas de trabajo total.

Fase	Descripción	Horas estimadas
Fase 1: Análisis	Investigación, diseño de base de datos, diagramas de arquitectura y de casos de uso.	40h
Fase 2: Backend	Configuración de AWS, configuración pipeline CI/CD, FastAPI, modelos y lógica de negocio.	80h
Fase 3: Frontend - Desktop	Desarrollo de la interfaz de fisioterapeutas (Pyside6 + Python).	70h
Fase 4: Frontend - Web	Desarrollo de la interfaz de pacientes (React + TypeScript).	80h
Fase 5: Finalización	Testing final, pruebas, despliegues y redacción de la memoria.	30h
		Total: 300h.



Planificación temporal real.

Basándome en los commits y el registro de actividad real, el tiempo dedicado se ajustó de esta forma:

Fase	Descripción	Horas reales
Fase 1: Análisis	Investigación, diseño de base de datos, diagramas de arquitectura y de casos de uso.	40h
Fase 2: Backend	Configuración de AWS, configuración pipeline CI/CD, FastAPI, modelos y lógica de negocio.	90h
Fase 3: Frontend - Desktop	Desarrollo de la interfaz de fisioterapeutas (Pyside6 + Python).	50h
Fase 4: Frontend - Web	Desarrollo de la interfaz de pacientes (React + TypeScript).	100h
Fase 5: Finalización	Testing final, pruebas, despliegues y redacción de la memoria.	20h
		Total: 300h.



Análisis de las desviaciones.

Aunque al final si se hayan cumplido las 300 horas, han existido variaciones durante el desarrollo.

- **Backend (+10h):** En esta fase ha habido un aumento de lo esperado debido a horas de configuración en AWS y a la realización por primera vez de un pipeline CI/CD del proyecto. Por otro lado, el desarrollo del backend con FastAPI ha sido ágil y satisfactorio.
- **Frontend – Desktop (-20h):** Estudiar en el grado de DAM el desarrollo de aplicaciones de escritorio gracias a la asignatura de Desarrollo de Interfaces me ha permitido realizar este Sprint de manera eficiente y más rápida de lo esperado, encontrando algunas dificultades en el desarrollo, pero no tanto lo esperado.
- **Frontend – Web (+ 20h):** Por otro lado, al no cursar el ciclo de Desarrollo de Aplicaciones Web (DAW) he encontrado más problemas para adaptarme al desarrollo web del proyecto. La realización de cursos para entender completamente React o mi falta de adaptación a TypeScript ha conllevado más tiempo de lo esperado en esta parte del proyecto.
- **Finalización (-10h):** Un desarrollo robusto gracias a la implementación de integridad continua, ha permitido una finalización del proyecto más ágil debido a la aparición de menos bugs de lo esperado.

Diagrama de Gantt.

Un diagrama de Gantt es un cronograma visual para hacer seguimiento de las tareas y los hitos a lo largo del ciclo de vida de un proyecto. (Meardon, s.f.).

A continuación, está el Diagrama de Gantt de este proyecto dividido en sus Sprints:

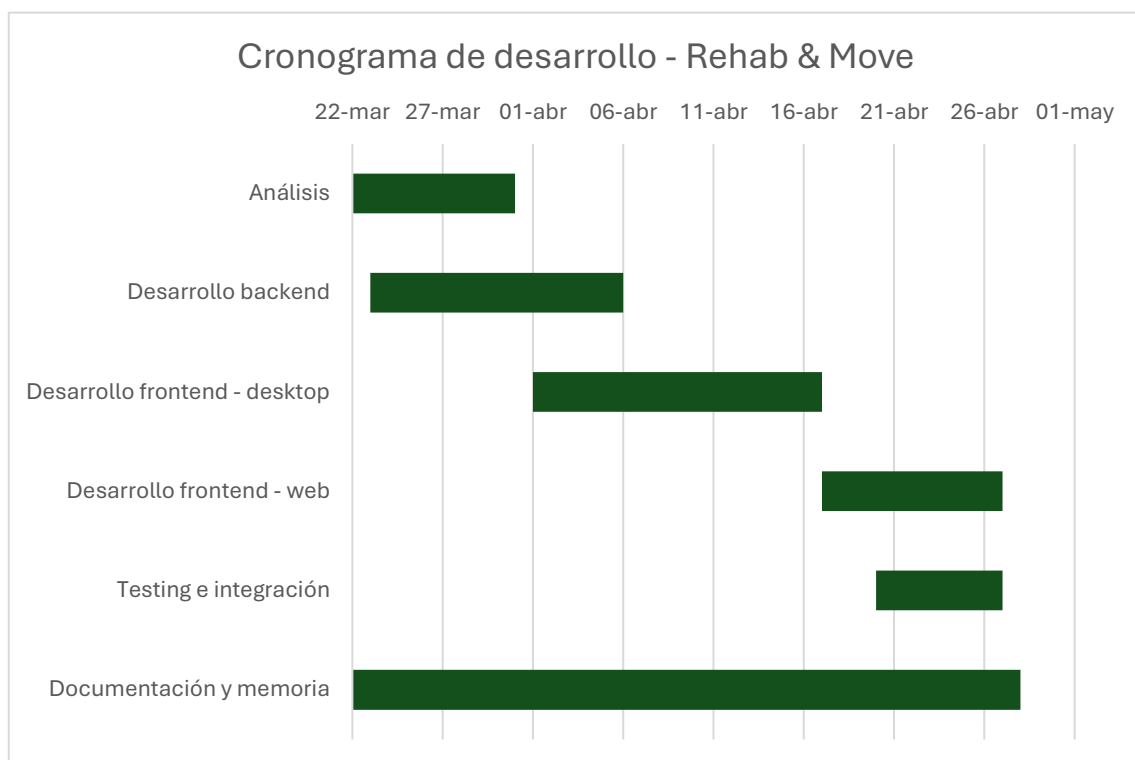


Diagrama de Gantt Rehab&Move.

VIABILIDAD DE NEGOCIO

Análisis de mercado y público objetivo

Esta aplicación está enfocada para un nicho concreto. El público objetivo son clínicas de fisioterapia medianas/pequeñas o fisioterapeutas autónomos que aún utilizan papel o Excel como método de gestión de citas.

Además, Rehab&Move cumple una necesidad: la falta de seguimiento real de ejercicios en casa y la ausencia de feedback en la evolución del dolor del paciente.

Análisis DAFO

El análisis DAFO (Debilidades, Amenazas, Fortalezas y Oportunidades) es una herramienta de planificación estratégica esencial para evaluar la situación interna y externa de una empresa o en este caso, de un proyecto. Permite identificar puntos fuertes y áreas de mejora (interno) frente a oportunidades y amenazas del mercado (externo) para tomar mejores decisiones. (Empresa., s.f.).

Fortalezas (interno):

- Arquitectura cloud disponible desde cualquier sitio (AWS).
- Especialización en métricas de dolor y seguimiento personalizado del paciente.

Debilidades (interno):

- Marca nueva sin presencia en el mercado.
- Dependencia de servicios de terceros (AWS / YouTube).

Oportunidades (externo):

- Adaptabilidad a pymes y conocimiento previo del sector.
- Auge de la telemedicina y digitalización en el sector de la salud.

Amenazas (externo):

- Competencia con software de gestión ya asentados.
- Falta de conocimiento legislativo frente a otras empresas para cumplimiento de normativas (RGPD).

Modelo de negocio: Monetización

Rehab&Move es una aplicación con backend en la nube y que requiere mantenimiento de forma regular, por ello, lo más lógico es que utilice como es que se ofrezca como SaaS (Software as Service).

La aplicación se puede ofrecer con un plan de suscripción mensual, que a su vez se puede dividir en dos planes:

- **Prueba gratuita (0€ / mes):** Ofrece un mes de suscripción premium gratuita para permitir que el cliente se adapte y facilitar la adquisición del producto en el mercado.
- **El plan básico (29€ / mes):** La aplicación tiene limitaciones, solo permite gestionar 100 pacientes como máximo y la funcionalidad de seguimiento de dolor no está disponible.
- **El plan premium (59€ / mes):** Ofrece todas las funcionalidades disponibles, además de gestión ilimitada de pacientes.

Por otro lado, al usar AWS el coste inicial es muy bajo, ya que se paga por uso, lo que permite que las pruebas gratuitas no generen grandes costes sobre todo al inicio de la puesta en el mercado.



Viabilidad técnica y económica

Viabilidad técnica

Las herramientas técnicas escogidas para esta aplicación son adecuadas y actualizadas al contexto actual de desarrollo de aplicaciones.

Infraestructura robusta (AWS)

Al utilizar Amazon Web Services, la viabilidad técnica es máxima ya que delegamos el mantenimiento del hardware, el suministro eléctrico y la seguridad física a Amazon. Además, el uso de AWS Elastic Beanstalk facilita el despliegue automático a través de GitHub Actions y la posible escalabilidad futura.

Stack tecnológico

Backend (FastAPI): Es un framework moderno y de alto rendimiento enfocado a construir APIs con Python 3.7+, destacado por su velocidad y su uso de tipado de Python, lo que asegura que la aplicación pueda manejar múltiples fisioterapeutas simultáneamente sin provocar problemas de rendimiento.

Frontend (PySide6 y React): El uso de estas tecnologías garantiza que cualquier desarrollador pueda dar soporte al proyecto en el futuro, al ser herramientas estandarizadas en el mercado de desarrollo de software.

Disponibilidad

Al encontrarse en la nube, la solución es técnica y operativamente viable desde cualquier lugar con conexión a internet, eliminando barreras geográficas.



Viabilidad económica

En este apartado calculo los costes de creación y mantenimiento de la aplicación.

Inversión inicial

- Horas de desarrollo estimadas: 300 horas totales, 250 horas de desarrollo técnico y 50 horas de investigación, diseño de diagramas y redacción de memoria.
- Precio / Hora Junior: 25 €/h.
- Coste de personal: 7.500 €.
- Hardware y Software: Se han utilizado herramientas Open Source (VS Code, Python, React), lo que quiere decir que el coste de licencias es de 0 €.

Costes Operativos Mensuales (Mantenimiento en AWS).

Gracias a los servicios en la nube, el coste es muy bajo sobre todo inicialmente.

AWS Free Tier:

Durante el primer año, la mayoría de servicios son gratuitos o de coste mínimo (Instance t3.micro).

Estimación post – Free Tier (para 100 usuarios):

- AWS RDS (Base de Datos): ~15 €/mes.
- AWS Elastic Beanstalk/EC2: ~12 €/mes.
- Tráfico de datos: ~2 €/mes.
 - Total coste fijo: ~29 €/mes.

Análisis de Rentabilidad (Punto de Equilibrio):

Si aplicamos el modelo SaaS comentado anteriormente, al obtener un solo cliente con el modelo de plan básico (30 €/mes) ya se cubren los costes operativos de AWS. A partir del segundo cliente, todo es beneficio para amortizar la inversión inicial del desarrollo.



Conclusión de Viabilidad de Negocio

Este proyecto presenta una gran viabilidad económica. Gracias al modelo de pago por uso de AWS y al uso de tecnologías de open source o código abierto, los costes fijos mensuales son mínimos y directamente relacionados con el crecimiento de sus ventas. Esto ofrece una rentabilidad con una base de clientes reducida, presentando un riesgo financiero básicamente nulo y un alto potencial de escalabilidad en el apartado técnico.

TRABAJOS FUTUROS

En cuanto al trabajo futuro y próximas mejoras, Rehab&Move podría mejorar de la siguiente manera:

- **Obligatoriedad de usar claves robustas:**

Me gustaría implementar en el futuro una serie de condiciones que obligue a cualquier usuario (fisioterapeuta o paciente) registrarse con una contraseña que cumpla los requisitos de seguridad de la ley LOPDGDD (Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y Garantía de Derechos Digitales).

- **Desarrollo completo para la parte de escritorio y la parte web:**

En este proyecto, he enfocado el desarrollo multiplataforma de la siguiente manera: un tipo de usuario para la aplicación de escritorio (fisioterapeuta) y otro tipo para la aplicación web (paciente).

Aunque, es interesante que el profesional tenga la aplicación de escritorio como herramienta de gestión, ya que puede estar instalada en las clínicas y, por otro lado, es más sencillo para el paciente acceder a través de una web a sus ejercicios asignados, enviar el feedback al profesional sanitario... Sin embargo, para ser una aplicación multiplataforma completa debería estar el desarrollo para los dos tipos de usuario en todos los medios de la aplicación.

Esto convertiría Rehab&Move en una solución omnicanal real.

- **Sistemas de categorización en los registros de dolor:**

Durante el desarrollo he notado que sería interesante tener un sistema de categorización de patologías para poder distinguir los registros de dolor de los pacientes.

No es lo mismo ver la evolución de hace un año por un dolor lumbar, que la evolución actual de un problema del hombro del mismo paciente, y actualmente la aplicación no distingue si los registros son para un problema u otro.

- **Refactorización de la tabla fisio y la tabla paciente (Usuarios):**

Aunque las entidades fisio y paciente tengan algunas columnas y relaciones diferentes, haría más sencillo el desarrollo el que las dos tablas hereden de una tabla común denominada usuarios, para facilitar el login y la gestión de permisos con un código más limpio.

- **Despliegue de Aplicación Móvil:**

Al ser una aplicación multiplataforma, el siguiente paso natural es el desarrollo móvil.

A través de tecnologías como Kotlin o React Native podría aprovechar el backend actual para desarrollar una app en Android o IOS, permitiendo al paciente registrar su dolor o ver sus videos incluso sin conexión a internet.

- **Uso de IA para análisis postural en tiempo real:**

Un salto en el desarrollo de la aplicación podría ser incorporar Inteligencia Artificial en Rehab&Move.

A través de Computer Vision (librería MediaPipe) se podría valorar gracias a la cámara de los dispositivos si el paciente está realizando el ejercicio de manera correcta, proporcionando feedback instantáneo.



CONCLUSIONES

La principal problemática era el abandono del tratamiento de fisioterapia de los pacientes, debido a una falta de adherencia en los ejercicios. Creo firmemente que **Rehab&Move** puede ser una herramienta de apoyo al fisioterapeuta para monitorizar la realización de la rehabilitación, además de gestor de pacientes, gestor de citas y visualización de la evolución del dolor.

Al ser una aplicación multiplataforma, se ha creado un sistema conectado que funciona desde una aplicación de escritorio (PySide6) hasta una web funcional (React). Además, todo está desplegado en la nube (AWS/Vercel) lo que permite que la información esté sincronizada en tiempo real y accesible desde cualquier punto.

Para finalizar, este proyecto me ha demostrado que el desarrollo es mucho más satisfactorio e incluso, más efectivo, cuando el desarrollador comprende profundamente el sector en el que está trabajando. Mi experiencia previa como sanitario me ha permitido diseñar una interfaz y una lógica enfocada a resolver los problemas que vivía en mi día a día.



ENLACES

Enlace a GitHub.

Repositorio donde se encuentra todo el código y todos los commits realizados:

https://github.com/davidprados99/Rehab-Move_Project

Enlace a YouTube.

Video oculto de YouTube en el que muestro la funcionalidad de la aplicación y explico la estructura del proyecto:

<https://www.youtube.com/watch?v=B9Y3DjwYBvU>



REFERENCIAS

- 4Geeks. (2026). *Todo lo que necesitas saber sobre SQLAlchemy*. Obtenido de 4geeks.com: <https://4geeks.com/es/lesson/todo-lo-necesario-para-empezar-usar-sqlalchemy>
- aplyca. (17 de Febrero de 2026). *¿Qué es Vercel?* Obtenido de aplyca.com: <https://www.aplyca.com/blog/blog-que-es-vercel-desarrollar-previsualizar-enviar>
- AWS. (2026). *¿Por qué utilizar IAM?* Obtenido de aws.amazon.com: <https://aws.amazon.com/es/iam/>
- AWS. (2026). *Amazon Relational Database Service*. Obtenido de aws.amazon.com: <https://aws.amazon.com/es/rds/>
- AWS. (2026.). *AWS Elastic Beanstalk*. Obtenido de aws.amazon.com: <https://aws.amazon.com/es/elasticbeanstalk/>
- Axios. (s.f.). *Axios*. Obtenido de Axios: <https://axios.rest/es/pages/getting-started/first-steps>
- Collins, E. (8 de Octubre de 2020). *passlib 1.7.4*. Obtenido de pypi.org: <https://pypi.org/project/passlib/#:~:text=Passlib%20is%20a%20password%20hashing,for%20managing%20existing%20password%20hashes.>
- Drumond, C. (s.f.). *¿Qué es scrum? Desglose del marco de trabajo ágil*. Obtenido de Atlassian: <https://www.atlassian.com/es/agile/scrum>
- Empresa., D. G. (s.f.). *Herramienta DAFO*. Obtenido de lpyme.org: <https://dafo.ipyme.org/Home>
- Gemini. (s.f.). Obtenido de Gemini: <https://gemini.google.com/app?hl=es-ES>
- GitHub. (s.f.). *Acerca de GitHub y Git*. Obtenido de github.com: <https://docs.github.com/es/get-started/start-your-journey/about-github-and-git>
- GitHub. (s.f.). *Descripción de GitHub Actions*. Obtenido de github.com: <https://docs.github.com/es/actions/get-started/understand-github-actions>
- Gómez, u. J. (s.f.). *Python requests. La librería para hacer peticiones http en Python*. Obtenido de j2logo.com: <https://j2logo.com/python/python-requests-peticiones-http/>
- Gunicorn. (s.f.). *Serve Python on the Web*. Obtenido de gunicorn.org: <https://gunicorn.org/>
- Huet, P. (21 de Enero de 2022). *OpenWebinars*. Obtenido de Qué es Tailwind CSS y por qué deberías usarlo: <https://openwebinars.net/blog/que-es-tailwind-css-y-por-que-deberias-usarlo/>



- Jackson, B. (1 de Octubre de 2025). *¿Qué Es Nginx y Cómo Funciona?* Obtenido de kinsta.com: <https://kinsta.com/es/blog/que-es-nginx/>
- KeepCoding. (16 de Abril de 2024). *¿Qué es Matplotlib y cómo funciona?* Obtenido de KeepCoding: <https://keepcoding.io/blog/que-es-matplotlib-y-como-funciona/>
- Lucidchart. (s.f.). *Qué es una diagrama entidad-relación*. Obtenido de Lucidchart: <https://www.lucidchart.com/pages/es/que-es-un-diagrama-entidad-relacion>
- Meardon, E. (s.f.). *¿Qué es un diagrama de Gantt? Funciones y beneficios clave con plantillas útiles*. Obtenido de Atlassian: <https://www.atlassian.com/es/agile/project-management/gantt-chart>
- Meta. (s.f.). *react.dev*. Obtenido de Crea interfaces de usuario a partir de componentes: <https://es.react.dev/>
- Microsoft. (s.f.). *¿Qué es PostgreSQL?* Obtenido de Microsoft: <https://azure.microsoft.com/es-es/resources/cloud-computing-dictionary/what-is-postgresql>
- Morales, A. (s.f.). *Descubre el nuevo pgAdmin 4 para trabajar con PostGIS*. Obtenido de MappingGIS: <https://mappinggis.com/2017/11/descubre-el-nuevo-pgadmin-4-para-trabajar-con-postgis/#:~:text=pgAdmin%20es%20una%20herramienta%20indispensable%20para%20gestionar,de%20c%C3%B3digo%20abierto%20m%C3%A1s%20avanzada%20del%20mundo>
- Oracle. (s.f.). *¿Qué es Python?* Obtenido de Oracle: <https://www.oracle.com/es/developer/what-is-python-for-developers/>
- Platzi. (10 de Agosto de 2020). *Que es dotenv? Para que sirve?* Obtenido de platzi.com: <https://platzi.com/blog/que-es-dotenv-para-que-sirve/>
- Pykes, K. (3 de Mayo de 2024). *Cómo utilizar Pytest para pruebas unitarias*. Obtenido de datacamp.com: <https://www.datacamp.com/es/tutorial/pytest-tutorial-a-hands-on-guide-to-unit-testing>
- Python GUIs. (19 de Mayo de 2025). *he complete PySide6 tutorial — Create GUI applications with Python*. Obtenido de Python GUIs: <https://www.pythonguis.com/pyside6-tutorial/>
- Q2BStudio. (19 de Enero de 2026). *Uvicorn en APIs modernas de Python*. Obtenido de q2bstudio.com: <https://www.q2bstudio.com/nuestro-blog/502494/uvicorn-potencia-apis-modernas-en-python?scriptcookies=1>
- Rodrigo Cubillos-Bravo, D. A.-S. (2022). *Tecnologías de apoyo a la rehabilitación e inclusión. Recomendaciones para el abordaje de niñas,*



niños y adolescentes con trastornos del neurodesarrollo. *Revista Médica Clínica Las Condes*, 604-614.

Salgado, L. G. (15 de Marzo de 2024). *¿Qué es Swagger UI?* Obtenido de keepcoding.io: <https://keepcoding.io/blog/que-es-swagger-ui/>

Tiangolo. (s.f.). *FastAPI*. Obtenido de Tiangolo: <https://fastapi.tiangolo.com/es/>

Tuychiev, B. (25 de Junio de 2025). *Pydantic: Una guía con ejemplos prácticos*. Obtenido de Datacamp.com: <https://www.datacamp.com/es/tutorial/pydantic>

Vite. (s.f.). *Vite*. Obtenido de Vite: <https://es.vite.dev/guide/>

Wondershare. (11 de Septiembre de 2025). *Cómo crear un diagrama de base de datos*. Obtenido de Wondershare: <https://edraw.wondershare.es/how-to-create-database-diagram.html>

Zúñiga, F. G. (22 de Abril de 2024). *Arsys*. Obtenido de TypeScript, el JavaScript de última generación: <https://www.arsys.es/blog/typescript-javascript>

Zúñiga, F. G. (17 de Julio de 2025). *¿Qué es Visual Studio Code y cuáles son sus ventajas?* Obtenido de Arsys: <https://www.arsys.es/blog/que-es-visual-studio-code-y-cuales-son-sus-ventajas>

Zúñiga, F. G. (17 de Julio de 2025). *Todo sobre la arquitectura cliente-servidor*. Obtenido de arsys.es: <https://www.arsys.es/blog/todo-sobre-la-arquitectura-cliente-servidor>